

SMART HOME AUTOMATION AND MONITORING SYSTEM

IoT-Based Project Using ESP32, Blynk Cloud and Sensors

PROJECT DOCUMENTATION

PROGRAMMING

C/C++ with Arduino Framework

IoT PLATFORM | SIMULATION

Blynk Cloud | Velxio

Architecture

Sensors → ESP32 → Wi-Fi → Blynk Cloud → Dashboard

Project Documentation Submitted by:
Divya Vinay Garol

Table of Contents

1. Introduction	3
2. Problem Statement.....	3
3. Objectives.....	3
4. System Architecture.....	3
5. Hardware Components	4
6. Pin Connections	4
7. Software and Technologies.....	5
8. Blynk Dashboard	6
9. Working Principle.....	6
10. Automation Logic.....	7
11. Simulation Using Velxio	7
12. Example Output	7
13. Advantages.....	8
14. Limitations.....	8
15. Future Scope	8
16. Real-World Implementation and Safety	8
17. Conclusion.....	9
18. Security Note.....	9
Appendix A: Complete Arduino Source Code	10

1. Introduction

The Smart Home Automation and Monitoring System is an IoT-based project developed using an ESP32 microcontroller, sensors, Wi-Fi communication and Blynk Cloud. The system monitors temperature, humidity, light intensity, motion and distance in real time. Based on sensor readings, it can automatically control a light, fan, door mechanism and buzzer. Blynk Cloud also provides remote monitoring and manual control through a dashboard.

2. Problem Statement

Traditional home appliances often require manual operation and provide limited real-time monitoring. The project aims to develop a smart, connected system that can sense the surrounding environment, make basic automatic decisions and allow the user to monitor and control devices remotely.

3. Objectives

- Monitor temperature and humidity using DHT11.
- Detect light intensity using an LDR/photoresistor.
- Detect human motion using a PIR sensor.
- Detect nearby objects or a person using an HC-SR04 ultrasonic sensor.
- Automatically control light, fan and door functions.
- Provide an audible alert using a buzzer.
- Send sensor information to Blynk Cloud through Wi-Fi.
- Provide manual control through the Blynk dashboard.
- Test the complete concept in the Velxio online simulator.

4. System Architecture



DHT11, LDR, PIR and HC-SR04 provide input to the ESP32. The ESP32 processes these readings and controls the light LED, fan LED, servo and buzzer. Blynk Cloud is used for remote monitoring and control.

5. Hardware Components

Component	Purpose
ESP32 DevKit	Main microcontroller and Wi-Fi communication
DHT11	Temperature and humidity measurement
LDR module	Light intensity sensing
PIR sensor	Motion detection
HC-SR04	Distance / nearby-person detection
Servo motor	Prototype door movement
LED	Prototype light and fan indicators
Buzzer	Audible alert

6. Pin Connections

Component	Connection
DHT11	VCC / GND / SIG → 3.3V / GND / GPIO 26
LDR	VCC / GND / A0 → 3.3V / GND / GPIO 34
PIR	+ / D / - → 3.3V / GPIO 27 / GND
HC-SR04	VCC / GND / TRIG / ECHO → VIN / GND / GPIO 5 / GPIO 18
Servo	V+ / GND / Signal → VIN / GND / GPIO 19
Light LED	GPIO 23, through 220Ω resistor
Fan LED	GPIO 22, through 220Ω resistor
Buzzer	+ / - → GPIO 21 / GND

8. Blynk Dashboard

The Blynk dashboard is used to display sensor values and provide manual controls. The planned virtual pins are:

Virtual Pin	Function
V0	Temperature
V1	Humidity
V2	Motion
V3	Light intensity
V4	Light control
V5	Fan control
V6	Door control

Live Dashboard

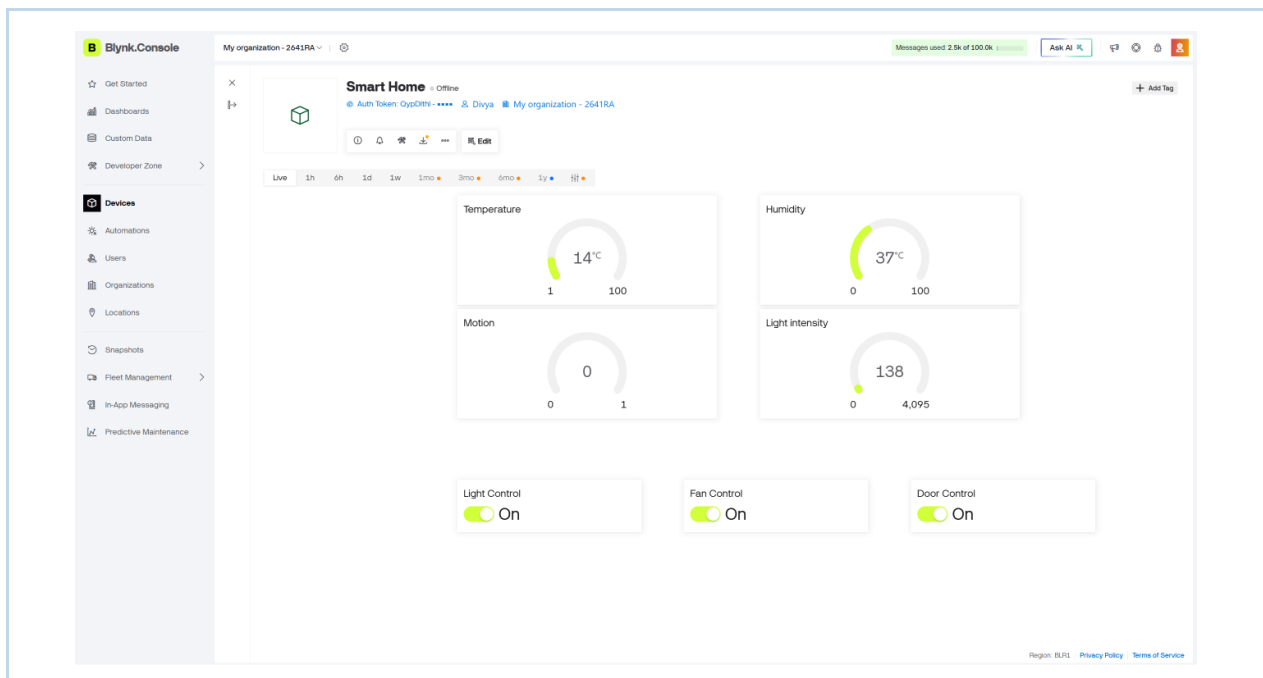


Fig. 2 — Blynk Cloud dashboard showing live temperature, humidity, light intensity, motion status and manual device controls.

9. Working Principle

The ESP32 continuously reads the connected sensors. DHT11 provides temperature and humidity, the LDR provides an analog light reading, the PIR reports motion status and the HC-SR04 measures distance. The ESP32 processes these readings and applies the programmed automation rules. At the same time, selected values are sent to Blynk Cloud over Wi-Fi for remote monitoring.

10. Automation Logic

- **Light:** When motion is detected and the environment is dark, the light LED is switched ON automatically.
- **Fan:** When the measured temperature is above 30°C, the fan LED is switched ON automatically.
- **Door:** When the measured distance is below 20 cm, the servo moves to the open position; otherwise it returns to the closed position.
- **Buzzer:** When a person/object is detected within 20 cm, the buzzer is switched ON as an alert.
- **Manual control:** Blynk controls can override automatic operation for the light, fan and door.

11. Simulation Using Velxio

The project is simulated using Velxio, an online electronics simulation environment. The ESP32 connects to the simulator's available Wi-Fi network and communicates with Blynk Cloud. The simulation allows the sensors, actuators, Wi-Fi communication and cloud dashboard to be tested without requiring physical hardware.

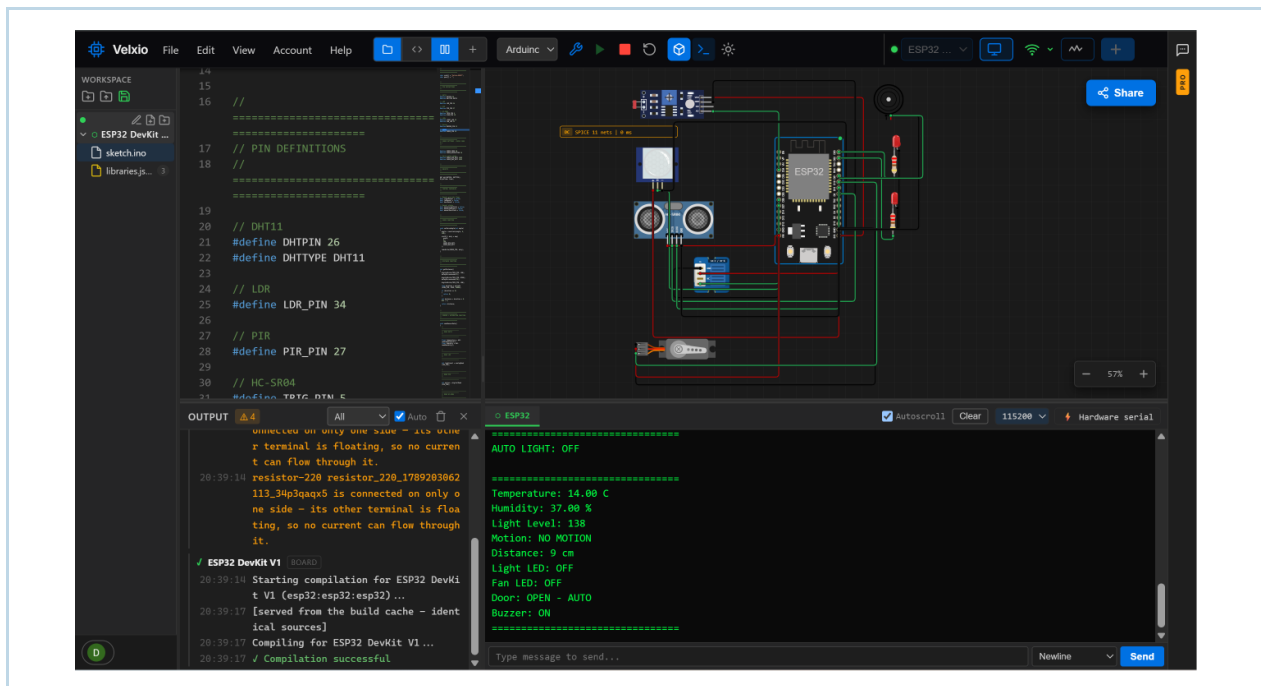


Fig. 3 — Velxio simulation workspace: circuit wiring, firmware code and live serial-monitor output during a successful test run.

12. Example Output

Example sensor readings observed during testing included temperature and humidity values from the DHT11, an analog light level from the LDR, motion status from the PIR, distance from the ultrasonic sensor, and ON/OFF states for the light, fan and buzzer. Blynk communication was also tested successfully for dashboard monitoring and control.

13. Advantages

- Real-time environmental monitoring.
- Automatic control reduces the need for continuous manual operation.
- Remote monitoring and control through Blynk Cloud.
- Combines multiple sensors and devices in one system.
- Can be expanded with additional IoT features.
- Simulation reduces hardware-testing cost during development.

14. Limitations

- The current implementation is a prototype/simulation.
- The 9g servo is suitable for a model door, not a full-size real door.
- LEDs represent appliances such as lights and fans in the prototype.
- Real AC appliances require properly rated relay/contactors and electrical protection.
- The automation thresholds may require calibration for a real environment.

15. Future Scope

- Add mobile notifications for security and abnormal conditions.
- Add energy monitoring and power-consumption analysis.
- Add voice-control integration.
- Add more rooms and additional sensors.
- Add improved door-lock hardware for real-world deployment.
- Add sensor-failure detection and stronger Wi-Fi reconnection handling.
- Develop a more advanced dashboard for historical data and analytics.

16. Real-World Implementation and Safety

For real deployment, the prototype LEDs can be replaced by properly rated relay or contactor circuits, and the model servo can be replaced by an appropriate electronic door-lock actuator. ESP32 GPIO pins must never be connected directly to 230V AC. A certified power supply, suitable protection, enclosure and proper electrical installation are required for mains-powered devices.

17. Conclusion

The Smart Home Automation and Monitoring System demonstrates how an ESP32 can combine sensing, automation, Wi-Fi communication and cloud-based monitoring. By integrating DHT11, LDR, PIR and HC-SR04 sensors with LEDs, a servo and a buzzer, the project provides a practical prototype of an IoT-enabled smart home. Blynk Cloud adds remote monitoring and control, while Velxio provides a convenient environment for simulation and testing.

18. Security Note

Security Note

Do not publish real Blynk authentication tokens, Wi-Fi passwords or other private credentials in the GitHub repository or project documentation. Use placeholders in shared code and documentation.

Appendix A: Complete Arduino Source Code

The firmware below (Smart_home_IoT.ino) runs on the ESP32 and implements sensor reading, automation logic and Blynk Cloud communication. Credential fields are left as placeholders, in line with the security note above.

```
#define BLYNK_TEMPLATE_ID "YOUR_BLYNK_TEMPLATE_ID"
#define BLYNK_TEMPLATE_NAME "Smart Home"
#define BLYNK_AUTH_TOKEN "YOUR_BLYNK_AUTH_TOKEN"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>

// =====
// Wi-Fi
// =====

char ssid[] = "Velxio-GUEST";
char pass[] = "";

// =====
// DHT11
// =====

#define DHTPIN 26
#define DHTTYPE DHT11

// =====
// Sensors
// =====

#define LDR_PIN 34
#define PIR_PIN 27

#define TRIG_PIN 5
#define ECHO_PIN 18

// =====
// Outputs
// =====

#define LIGHT_LED 23
#define FAN_LED 22
#define BUZZER_PIN 21

// =====
// Servo
// =====

#define SERVO_PIN 19

#define SERVO_FREQ 50
#define SERVO_RESOLUTION 16

#define SERVO_MIN_DUTY 1638
#define SERVO_MAX_DUTY 8192

// =====
// Objects
// =====

DHT dht(DHTPIN, DHTTYPE);
```

```

BlynkTimer timer;

// =====
// Manual Control Flags
// =====

bool lightManual = false;
bool fanManual = false;
bool doorManual = false;

bool manualLightState = false;
bool manualFanState = false;
bool manualDoorState = false;

// =====
// Servo Function
// =====

void setServoAngle(int angle)
{
    angle = constrain(angle, 0, 180);

    uint32_t duty = map(
        angle,
        0,
        180,
        SERVO_MIN_DUTY,
        SERVO_MAX_DUTY
    );

    ledcWrite(SERVO_PIN, duty);
}

// =====
// Read Sensors and Control System
// =====

void sendSensorData()
{
    // -----
    // Read DHT11
    // -----

    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    // -----
    // Read LDR and PIR
    // -----

    int lightLevel = analogRead(LDR_PIN);
    int motion = digitalRead(PIR_PIN);

    // -----
    // Read HC-SR04
    // -----

    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);

    digitalWrite(TRIG_PIN, LOW);

```

```
long duration = pulseIn(ECHO_PIN, HIGH, 30000);

int distance = 0;

if (duration > 0)
{
    distance = duration * 0.034 / 2;
}

// -----
// Automatic Light
// -----

bool dark = lightLevel < 1500;

if (!lightManual)
{
    if (motion == HIGH && dark)
    {
        digitalWrite(LIGHT_LED, HIGH);
    }
    else
    {
        digitalWrite(LIGHT_LED, LOW);
    }
}

// -----
// Automatic Fan
// -----

if (!fanManual)
{
    if (!isnan(temperature) && temperature > 30)
    {
        digitalWrite(FAN_LED, HIGH);
    }
    else
    {
        digitalWrite(FAN_LED, LOW);
    }
}

// -----
// Automatic Door
// -----

if (!doorManual)
{
    if (distance > 0 && distance < 20)
    {
        setServoAngle(90);
    }
    else
    {
        setServoAngle(0);
    }
}

// -----
// Buzzer
// -----

if (distance > 0 && distance < 20)
{
    digitalWrite(BUZZER_PIN, HIGH);
}
```

```

else
{
    digitalWrite(BUZZER_PIN, LOW);
}

// =====
// Serial Monitor
// =====

Serial.println();
Serial.println("=====");

if (!isnan(temperature))
{
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" C");
}
else
{
    Serial.println("Temperature: ERROR");
}

if (!isnan(humidity))
{
    Serial.print("Humidity: ");
    Serial.print(humidity);
    Serial.println(" %");
}
else
{
    Serial.println("Humidity: ERROR");
}

Serial.print("Light Level: ");
Serial.println(lightLevel);

Serial.print("Motion: ");

if (motion == HIGH)
{
    Serial.println("MOTION DETECTED");
}
else
{
    Serial.println("NO MOTION");
}

Serial.print("Distance: ");
Serial.print(distance);
Serial.println(" cm");

Serial.print("Light: ");

if (digitalRead(LIGHT_LED) == HIGH)
{
    Serial.println("ON");
}
else
{
    Serial.println("OFF");
}

Serial.print("Fan: ");

if (digitalRead(FAN_LED) == HIGH)
{
    Serial.println("ON");
}

```

```

    }
    else
    {
        Serial.println("OFF");
    }

    Serial.print("Door: ");

    if (doorManual)
    {
        if (manualDoorState)
        {
            Serial.println("OPEN - MANUAL");
        }
        else
        {
            Serial.println("CLOSED - MANUAL");
        }
    }
    else
    {
        if (distance > 0 && distance < 20)
        {
            Serial.println("OPEN - AUTOMATIC");
        }
        else
        {
            Serial.println("CLOSED - AUTOMATIC");
        }
    }
}

Serial.print("Buzzer: ");

if (digitalRead(BUZZER_PIN) == HIGH)
{
    Serial.println("ON");
}
else
{
    Serial.println("OFF");
}

Serial.println("=====");

// =====
// Send Data to Blynk
// =====

if (!isnan(temperature))
{
    Blynk.virtualWrite(V0, temperature);
}

if (!isnan(humidity))
{
    Blynk.virtualWrite(V1, humidity);
}

Blynk.virtualWrite(V2, motion);
Blynk.virtualWrite(V3, lightLevel);
}

// =====
// Blynk Light Control - V4
// =====

BLYNK_WRITE(V4)
{

```

```

int value = param.asInt();

lightManual = true;
manuallightState = value;

if (value == 1)
{
  digitalWrite(LIGHT_LED, HIGH);
  Serial.println("Blynk Light Control: ON");
}
else
{
  digitalWrite(LIGHT_LED, LOW);
  Serial.println("Blynk Light Control: OFF");
}
}

// =====
// Blynk Fan Control - V5
// =====

BLYNK_WRITE(V5)
{
  int value = param.asInt();

  fanManual = true;
  manualFanState = value;

  if (value == 1)
  {
    digitalWrite(FAN_LED, HIGH);
    Serial.println("Blynk Fan Control: ON");
  }
  else
  {
    digitalWrite(FAN_LED, LOW);
    Serial.println("Blynk Fan Control: OFF");
  }
}

// =====
// Blynk Door Control - V6
// =====

BLYNK_WRITE(V6)
{
  int value = param.asInt();

  doorManual = true;
  manualDoorState = value;

  if (value == 1)
  {
    setServoAngle(90);
    Serial.println("Blynk Door Control: OPEN");
  }
  else
  {
    setServoAngle(0);
    Serial.println("Blynk Door Control: CLOSED");
  }
}

// =====
// Setup
// =====

void setup()

```

```

{
  Serial.begin(115200);

  // -----
  // Sensor Setup
  // -----

  dht.begin();

  pinMode(PIR_PIN, INPUT);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  // -----
  // Output Setup
  // -----

  pinMode(LIGHT_LED, OUTPUT);
  pinMode(FAN_LED, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(LIGHT_LED, LOW);
  digitalWrite(FAN_LED, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  // -----
  // Servo Setup
  // -----

  ledcAttach(
    SERVO_PIN,
    SERVO_FREQ,
    SERVO_RESOLUTION
  );

  setServoAngle(0);

  // -----
  // Start Message
  // -----

  Serial.println();
  Serial.println("=====");
  Serial.println("      SMART HOME IoT");
  Serial.println("=====");

  Serial.println("Starting Blynk...");

  // -----
  // Connect to Blynk
  // -----

  Blynk.begin(
    BLYNK_AUTH_TOKEN,
    ssid,
    pass
  );

  Serial.println("Blynk connected!");

  // -----
  // Sensor update interval
  // -----

```

```
    timer.setInterval(  
        2000L,  
        sendSensorData  
    );  
}  
  
// =====  
// Main Loop  
// =====  
  
void loop()  
{  
    Blynk.run();  
    timer.run();  
}
```